

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Факультет прикладной математики, информатики и механики
Кафедра программного обеспечения
и администрирования информационных систем

Двусвязные списки

Курсовая работа
Направление 02.03.03 Математическое обеспечение
и администрирование информационных систем

Зав. кафедрой _____	д. ф.-м. н., проф. Артёмов М. А.
Обучающийся _____	1 курс, 91 группа, Миа Музамил Ахмад.
Руководитель _____	д. ф.-м. н., проф. Артёмов М. А.

Воронеж 2024

Аннотация

Данная работа посвящена изучению некоторых особенностей работы с двусвязными списками в языке C++.

Ее цель — исследовать возможности, которые представлены данным языком для обработки двусвязных списков, и реализовать программу, которая будет обеспечивать выполнение основных действий с двусвязными списками посредством использования созданной библиотеки.

Тема данной работы позволяет изучить основные принципы и идеи структурного программирования на примере языка C++.

СОДЕРЖАНИЕ

Аннотация	2
Введение.....	5
Концепции связанного списка.....	6
1. Преимущества связанного списка перед массивами.....	6
1.2 Недостатки связанного списка по сравнению с массивами	7
1.3. Предполагаемые структуры данных	8
1.4 Приложения связанных списков.....	9
1.5 Типы связанных списков	9
2. Двойной связанный список.....	10
2.1 Основные понятия о двусвязных списках	11
2.2 Преимущества и недостатки	11
Преимущества перед односвязным списком.....	11
Недостатки по сравнению с односвязным списком	12
2.3 Основные операции.....	12
2.4 Создание двусвязного списка	12
2.5 Вставка в двусвязный список.....	13
2.7.1 Вставка в начало списка	13
2.6 Удаление в двусвязном списке	16
2.7 Обход в двусвязных списках.....	19
3.Задачи	20
3.1 Создание двусвязного списка из файла и печать.....	20

3.2 Удаление всех записей о книгах, опубликованных до указанной даты.	21
Заключение	27
Список литературы	29

Введение

Данная работа посвящена изучению работы с двусвязными списками посредством использования созданной библиотеки в языке C++.

Двусвязный список – это тип данных, представляющих собой последовательность узлов, где каждый узел содержит два указателя: один указывает на следующий узел в последовательности, а другой – на предыдущий. В отличие от односвязного списка, где каждый узел имеет только один указатель на следующий узел, двусвязный список позволяет перемещаться по узлам как вперед, так и назад, что обеспечивает большую гибкость при обработке данных.

Тема данной работы имеет важное учебное значение и позволяет изучить основные принципы и идеи структурного программирования на примере выбранного языка.

1. Концепции связанного списка

Связанный список — это непоследовательный набор элементов данных. Это динамическая структура данных. Для каждого элемента данных в связанном списке существует связанный указатель, который указывает местоположение в памяти следующего элемента данных в связанном списке.

Элементы данных в связанном списке не находятся в последовательных ячейках памяти. Они могут быть где угодно, но доступ к этим элементам данных проще, поскольку каждый элемент данных содержит адрес следующего элемента данных.

1.1 Преимущества связанного списка перед массивами

Двусвязные списки (или двунаправленные списки) представляют собой тип линейной структуры данных, где каждый элемент (узел) содержит ссылки на предыдущий и следующий элементы. Это позволяет эффективно перемещаться по списку в обоих направлениях. В отличие от односвязных списков, в которых каждый элемент содержит только одну ссылку на следующий элемент, двусвязные списки обеспечивают большую гибкость и возможность более эффективного выполнения ряда операций, таких как удаление и вставка узлов.

Ниже приведены некоторые преимущества связанного списка перед массивами в программировании, в том числе:

Динамический размер: Одним из наиболее значительных преимуществ связанного списка перед массивами является то, что связанные списки могут динамически увеличиваться или уменьшаться во время выполнения. Это означает, что размер связанного списка можно регулировать для размещения новых элементов или удаления существующих элементов без необходимости выделять или освобождать блок памяти фиксированного размера, как в случае с массивами.

Эффективная вставка и удаление: Связанные списки позволяют эффективно вставлять и удалять элементы в любой позиции списка, тогда как массивы требуют смещения элементов при добавлении или удалении нового элемента, что может быть медленным и неэффективным для больших массивов.

Эффективность памяти: Связанные списки используют память более эффективно, чем массивы. В массиве все элементы должны храниться в смежных ячейках памяти, даже если некоторые элементы не используются. Напротив, связанные списки выделяют память только для используемых элементов, что может сэкономить память в тех случаях, когда размер набора данных неизвестен или меняется со временем.

Простая реализация абстрактных типов данных. Связанные списки легко использовать и реализовывать при реализации абстрактных типов данных, таких как стеки, очереди и деревья. Эти структуры данных требуют частой вставки и удаления элементов, и для этой задачи хорошо подходят связанные списки.

Более эффективная сортировка: В некоторых случаях связанные списки могут быть более эффективными для алгоритмов сортировки, чем массивы. Это связано с тем, что связанные списки не требуют замены элементов, таких как массивы, что может занять много времени для больших массивов.

Короче говоря, существует несколько преимуществ связанного списка перед массивами, таких как динамический размер, эффективная вставка и удаление, эффективность использования памяти, простая реализация абстрактных типов данных и в некоторых случаях более эффективная сортировка.

1.2 Недостатки связанного списка по сравнению с массивами

Несмотря на то, что связанный список имеет несколько преимуществ перед массивами, у него также есть некоторые недостатки, которые необходимо учитывать.

Произвольный доступ: Связанные списки не обеспечивают произвольный доступ к элементам, как это делают массивы. Чтобы получить доступ к элементу в связанном списке, нам нужно начать с начала списка и перемещаться по нему, пока не найдем нужный элемент. Это делает доступ к отдельным элементам связанного списка медленнее, чем в массиве.

Использование дополнительной памяти: Связанные списки требуют дополнительной памяти по сравнению с массивами. Каждый элемент связанного списка требует ссылки на следующий элемент, что занимает дополнительное пространство памяти. Напротив, массивам нужна память только для хранения самих элементов.

Более сложная реализация: Реализация связанного списка более сложна, чем реализация массива, поскольку требует управления указателями и динамического выделения памяти. Эта сложность может привести к увеличению количества ошибок и ошибок в коде.

Короче говоря, у связанного списка есть несколько недостатков по сравнению с массивами, такие как более медленное время доступа, дополнительное использование памяти и более сложная реализация.

1.3. Предполагаемые структуры данных

Для корректной работы с информацией, которая хранится в двусвязном списке необходимы вспомогательные структуры данных. Необходим тип, который будет обозначать структуру, являющуюся элементом двусвязного списка.

Так как рабочий файл будет хранить информацию о файлах, то структура с данными об одном файле будет иметь следующие поля:

1. Каталог — символьная строка длиной не более 1000 символов.
2. Название файла — символьная строка длиной не более 257 символов.
3. Расширение — символьная строка длиной не более 5 символов.
4. Дата создания — массив из 3 целых чисел.

5. Размер — целое число.

Данная программа не будет изменять двусвязный список непосредственно, она будет сначала считывать данные в список, а после решит поставленную задачу и выведет ответ в консоль.

1.4 Приложения связанных списков

- Динамическое распределение памяти
- Реализовано в стеке и очереди
- При отмене функциональности программного обеспечения
- Хеш-таблицы, графики
- Связанные списки используются для представления полиномов и управления ими. Полиномы — это выражения, содержащие члены с ненулевыми коэффициентами и показателями.

Например:

$$P(x) = a_0 X^n + a_1 X^{n-1} + \dots + a_{n-1} X + a_n$$

1.5 Типы связанных списков

По сути, мы можем поместить связанные списки в следующие четыре элемента:

1. Односвязный список.
2. Двойной связанный список.
3. Круговой связанный список.
4. Круговой двусвязный список.

2. Двойной связанный список

Двусвязные списки широко используются в программировании благодаря их эффективности и простоте реализации. Они используются в различных задачах, включая управление памятью, реализацию структур данных и работу с большими наборами данных.

Двойной связанный список — это двусторонний список, в котором все узлы имеют две ссылки. Это помогает получить доступ как к узлу-предшественнику, так и к узлу-последователю из заданной позиции узла. Он обеспечивает двунаправленное перемещение. Каждый узел содержит три поля:

- **Левая ссылка (previous)**
- **Данные**
- **Правая ссылка (next)**

Левая ссылка указывает на узел-предшественник, а правая ссылка указывает на узел-последователь. Поле данных хранит необходимые данные.

Многие приложения требуют поиска вперед и назад по узлам списка. Например, для поиска имени в телефонном справочнике потребуется сканирование вперед и назад по определенной области всего списка.

В одном связанном списке каждый узел имеет ссылку на следующий узел в последовательности. Таким образом, мы можем перейти от одного узла к другому только в одном направлении и не можем вернуться обратно. Мы можем решить эту проблему, используя двусвязный список. Двойной связанный список можно определить следующим образом...

Двойной связанный список — это последовательность элементов, в которой каждый элемент имеет ссылки на предыдущий элемент и следующий элемент последовательности.

В двусвязном списке каждый узел имеет ссылку на предыдущий и следующий узел. Таким образом, мы можем двигаться вперед, используя следующее поле, и можем двигаться назад, используя предыдущее поле.

2.1 Основные понятия о двусвязных списках

Узел двусвязного списка: Каждый узел двусвязного списка содержит три основных компонента:

1. **Данные (полезная нагрузка):** информация, которую хранит узел. Это могут быть любые данные, такие как числа, строки или более сложные объекты.
2. **Ссылка на предыдущий узел (prev):** указатель на предыдущий элемент списка. Если узел первый в списке, эта ссылка будет нулевой.
3. **Ссылка на следующий узел (next):** Указатель на следующий элемент списка. Если узел является последним в списке, эта ссылка будет нулевой.

Head: ссылка на первый узел списка. Если список пуст, head имеет значение nullptr.

Хвост (tail): ссылка на последний узел списка. Если список пуст, хвост имеет значение nullptr.

- В двусвязном списке первый узел всегда должен указываться головой.
- Всегда предыдущее поле головного узла должно быть NULL.
- Всегда следующее поле последнего узла должно быть NULL.

2.2 Преимущества и недостатки

Преимущества перед односвязным списком

- Двусвязный список можно перемещать как в прямом, так и в обратном направлении.
- Если задан указатель на удаляемый узел, то операция удаления в двусвязном списке более эффективна.

- Вставка нового узла перед данным узлом более эффективна.

Недостатки по сравнению с односвязным списком

1. Для хранения предыдущего указателя требуется дополнительное пространство.
2. Все операции двусвязного списка требуют сохранения дополнительного предыдущего указателя. Нам нужно изменить предыдущие указатели вместе со следующим указателем.

2.3 Основные операции

➤ Создание

➤ Вставка

- В начало списка
- Между двумя узлами
 - После данного узла
 - Перед данным узлом.
- В конец списка

➤ Удаление

- С начала списка
- С конца списка
- В заданной позиции

➤ Обход в списке

2.4 Создание двусвязного списка

```
struct node {  
    int data;  
    struct node* next;  
    struct node* prev;  
};  
struct node* head = NULL;  
struct node* tail = NULL;
```

2.5 Вставка в двусвязный список

Вставка нового узла в двусвязный список очень похожа на вставку нового узла в связанный список. Для поддержания связи предыдущего узла требуется небольшая дополнительная работа. Узел можно вставить в двусвязный список четырьмя способами.

2.7.1 Вставка в начало списка

Чтобы вставить новый узел в начало двойного списка, мы можем использовать следующие шаги:

- Выделите память для **new_node** и присвойте предоставленное значение его полю данных.
- Установите предыдущий указатель **new_node** на nullptr.
- Если список пуст:
 - Установите следующий указатель **new_node** на nullptr.
 - Обновите указатель головы, чтобы он указывал на **new_node**.
- 4. Если список не пуст:
 - Установите следующий указатель **new_node** на текущую голову.
 - Обновите предыдущий указатель текущей головы, чтобы он указывал на **new_node**.
 - Обновите указатель головы, чтобы он указывал на **new_node**.

```
void Add_Beginning(PtrNode ptr, int new_data)
{
    PtrNode new_node = new Node();
    new_node->data = new_data;
    new_node->next = ptr;
    new_node->prev = NULL;
    if (ptr)
        ptr->prev = new_node;
    ptr = new_node;
}
```

2.5.1 Добавьте узел после заданного узла

Нам дается указатель на узел как **prev_node**, и новый узел вставляется после данного узла. Это можно сделать, выполнив следующие шаги:

- Сначала создайте новый узел (**new_node**).
- Теперь вставьте данные в новый узел.
- Укажите следующий элемент **new_node** на следующий элемент **prev_node**.
- Укажите следующий из **prev_node** на **new_node**.
- Укажите предыдущий элемент **new_node** на **prev_node**.
- Укажите предыдущий или следующий из **new_node** на **new_node**.

```
void insert_after(ptrNODE ptr, TInfo elem)
{
    ptrNODE p = new DNODE(elem, ptr-> next, ptr);
    if (ptr == tail)
        tail = p;
    else
        ptr->next->prev = p;
    ptr->next = p;
}
```

Пусть указатель на этот узел будет **next_node**. Это можно сделать, выполнив следующие шаги:

- Выделите память для нового узла, пусть он будет называться **new_node**.
- Поместите данные в **new_node**.
- Установите предыдущий указатель этого **new_node** как предыдущий узел **next_node**.
- Установите предыдущий указатель **next_node** как **new_node**.
- Установите следующий указатель этого **new_node** как **next_node**.

- Установите следующий указатель предыдущего элемента **new_node** на **new_node**.

```
void indert_before(const ptrNODE ptr, TInfo elem)
{
    ptrNODE p = new DNODE(elem, ptr, ptr->prev);
    if (ptr == head)
        head = p;
    else
        ptr->prev->next = p;
    ptr->prev = p;
}
```

2.5.2 Вставка в конце

Новый узел всегда добавляется после последнего узла данного связанного списка. Это можно сделать, выполнив следующие шаги:

- Создайте новый узел (**new_node**).
- Поместите значение в новый узел.
- Сделайте следующий указатель **new_node** нулевым.
- Если список пуст, сделайте **new_node** головным.
- В противном случае перейдите в конец связанного списка.
- Теперь сделайте так, чтобы следующий указатель последнего узла указывал на **new_node**.
- Измените предыдущий указатель **new_node** на последний узел списка.

```
void add_last(PtrNode ptr, int new_data)
{
    PtrNode new_node = new Node();
    PtrNode last = ptr;
    new_node->data = new_data;
    new_node->next = NULL;

    if (ptr == NULL) {
        new_node -> prev = NULL;
```

```

        ptr = new_node;
        return;
    }
    while (last -> next )
        last = last->next;
    last->next = new_node;
    new_node->prev = last;
}

```

2.6 Удаление в двусвязном списке

В двусвязном списке нам нужно удалить узел из связанного списка. нам просто нужно скопировать указатель головы в указатель **ptr** и переместить указатель головы на следующий.

2.6.1 Удаление с начала списка

Удаление в двусвязном списке в начале — самая простая операция. Нам просто нужно скопировать указатель головы в указатель **ptr** и сдвинуть указатель головы к следующему.

АЛГОРИТМ

Шаг 1: если **head = null**

Написать нижнее положение

Перейдите к шагу 6

Шаг 2: установите **ptr = head**

Шаг 3: установить **head = head → next**

Шаг 4: установить **head → prev = Null**

Шаг 5: бесплатный **ptr**

Шаг 6: выход

2.6.2 Удаление из конца списка

Удаление последнего узла в двусвязном списке требует обхода списка, чтобы достичь последнего узла списка, а затем внести корректировки указателя в этой позиции.

Чтобы удалить последний узел списка, нам необходимо выполнить следующие шаги.

- Если список уже пуст, то условие `head == NULL` станет истинным и, следовательно, операция не может быть продолжена.
- Если в списке только один узел, то условие `head → next == NULL` становится истинным. В этом случае нам просто нужно присвоить заголовку списка значение `NULL` и освободить заголовок, чтобы полностью удалить список.
- В противном случае просто пройдите по списку, чтобы добраться до последнего узла списка. Это будет сделано с помощью следующих операторов.

```
.ptr = head;
.    if(ptr->next != NULL)
.    {
.        ptr = ptr -> next;
.    }
```

- Ptr будет указывать на последний узел list в конце цикла for. Просто сделайте следующий указатель предыдущего узла ptr равным `NULL`.

Step 1: IF HEAD = NULL

Write UNDERFLOW

Go to Step 7 [END OF IF]

Step 2: SET TEMP = HEAD

Step 3: REPEAT STEP 4 WHILE TEMP->NEXT != NULL

Step 4: SET TEMP = TEMP->NEXT

[END OF LOOP]

Step 5: SET TEMP ->PREV-> NEXT = NULL

Step 6: FREE TEMP

Step 7: EXIT

2.6.3 Удаление в заданной позиции

Учитывая односвязный список и позицию, удалите узел связанного списка в данной позиции.

АЛГОРИТМ

1. Входные данные: указатель на головной узел связанного списка и значение, которое нужно удалить.
2. Если связанный список пуст, верните NULL.
3. Если удаляемый узел является головным узлом, установите в качестве головного узла следующий узел и удалите исходный головной узел.
4. В противном случае пройдите по связанному списку от головного узла до тех пор, пока не будет найден узел, который нужно удалить.
5. Если узел, который нужно удалить, не найден, верните NULL.
6. В противном случае установите указатель следующего узла предыдущего узла на узел, следующий за удаляемым узлом.
7. Удалите узел, который нужно удалить.
8. Возвращает головной узел связанного списка.

```
void deleteNode(ptrNode ptr, int position)
{
    if (ptr == NULL)
        return;
    ptrNode temp = ptr;
    if (position == 0) {
        ptr = temp->next;
        free(temp);
        return;
    }
    for (int i = 0; temp != NULL && i < position - 1; i++)
        temp = temp->next;
    if (temp == NULL || temp->next == NULL)
```

```

        return;
    ptrNode next = temp->next->next;
    free(temp->next); // Free memory
    temp->next = next;
}

```

2.7 Обход в двусвязных списках

Обход — наиболее распространенная операция для каждой структуры данных. Для этого скопируйте указатель головы в любой из временных указателей ptr.

```
ptr=head;
```

затем пройдите по списку, используя цикл while. Продолжайте сдвигать значение переменной указателя ptr, пока не найдем последний узел. Последний узел содержит ноль в своей следующей части.

```

while (ptr)
{
    Std::cout<<ptr->data;
    ptr=ptr->next;
}

```

Однако перемещение означает посещение каждого узла списка один раз для выполнения какой-то конкретной операции. Здесь мы печатаем данные, связанные с каждым узлом списка.

Алгоритм

- Шаг 1: если head == null
напишите «недополнение»
перейдите к шагу 6
[конец если]
- Шаг 2. Установите ptr = head.
- Шаг 3. Повторите шаги 4 и 5, пока ptr != null.
- Шаг 4: напишите ptr → data
- Шаг 5: ptr = ptr → next
- Шаг 6: выход

3.Задачи

3.1 Создание двусвязного списка из файла и печать

```

#include<iostream>
#include<fstream>
#include<functional>
using TInfo = int;

struct DNODE
{
    TInfo info;
    DNODE* next, * prev;
    DNODE(TInfo info, DNODE* next = nullptr, DNODE* prev = nullptr)
    :info(info), next(next), prev(prev) {}
    ~DNODE()
    {
        next = nullptr;
        prev = nullptr;
    }
};

using ptrNODE=DNODE *;
struct DLIST
{
private:
    ptrNODE head, tail;
    size_t size;
public:
    DLIST() { head = nullptr; tail = nullptr; }
    DLIST(const char* file_name);
    void first_node(TInfo elem);
    void insert_after(ptrNODE ptr, TInfo elem);
    void print(std::ostream& stream = std::cout);
};

int main()
{
    DLIST list ("input.txt");
    list.print();
    return 0;
}

void DLIST::first_node(TInfo elem)
{
    head = new DNODE(elem);
    tail = head;
}

void DLIST::insert_after(ptrNODE ptr, TInfo elem)
{
    ptrNODE p = new DNODE(elem, ptr->next, ptr);
    if (ptr == tail)
        tail = p;
    else
        ptr->next->prev = p;
}

```

```

        ptr->next = p;
    }
DLIST::DLIST(const char* file_name)
{
    std::ifstream file(file_name);
    TInfo elem;
    file >> elem;
    first_node(elem);
    while (file >> elem)
        insert_after(tail, elem);
    file.close();
}

void DLIST::print(std::ostream& stream)
{
    ptrNODE p = head;
    while (p)
    {
        stream << p->info;
        p = p->next;
        std::cout << " ";
    }
}

```

3.2 Удаление всех записей о книгах, опубликованных до указанной даты.

Наша структура состоит из следующих элементов:

- Автор
- Название
- Год издания
- Количество страниц
- Специальность

```

#include<Windows.h>
#include<iostream>
#include<fstream>
#include<functional>
const int m = 50;
struct Book
{
private:
    char author[m];

```

```

        char title[m];
        int year;
        int pages;
        char specialty[m];
public:
    Book() {}
    Book(std::ifstream& file);
    void print();
    int get_year()
    {
        return year;
    }
};
using TInfo = Book *;
using ptrBook = Book *;
struct DNODE
{
    TInfo info;
    DNODE* next, * prev;
    DNODE(TInfo info, DNODE* next = nullptr, DNODE* prev = nullptr)
:info(info), next(next), prev(prev) {}
    ~DNODE()
    {
        next = nullptr;
        prev = nullptr;
    }
};
using ptrDNODE = DNODE *;
struct DLIST
{
private:
    ptrDNODE head, tail;
    size_t size;

public:
    DLIST() { head = nullptr, tail = nullptr; }
    DLIST(std::ifstream& file);
    DLIST(std::ifstream& file, std::function<int(Book, Book)>compare);
    TInfo get_elem(const ptrDNODE ptr);
    void first_node(TInfo elem);
    ptrDNODE get_head();
    void insert_after(ptrDNODE ptr, TInfo elem);
    void insert_before(ptrDNODE ptr, TInfo elem);

```

```

        void remove(ptrDNODE& ptr);
        void print();
};

void Task(DLIST& list, int year)
{
    ptrDNODE ptr_task = list.get_head();
    while (ptr_task && ptr_task->info->get_year() < year)
        list.remove(ptr_task);
}

int main()
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    std::ifstream file("input.txt");
    std::function <int(Book, Book)> compare_place = [] (Book a, Book b)
    {
        return a.get_year() < b.get_year();
    };
    if (file)
    {
        DLIST dlist(file, compare_place);
        dlist.print();
        std::cout << "*****\n";

        int year;
        std::cout << "Enter the year: ";
        std::cin >> year;
        Task(dlist, year);
        dlist.print();
    }
    else
        std::cout << "Not Exist\n";
    std::cin.get();
    return 0;
}

TInfo DLIST::get_elem(const ptrDNODE ptr)
{
    return (ptr ? ptr->info : nullptr);
}

void DLIST::first_node(TInfo elem)
{
    head = new DNODE(TInfo(elem));
    tail = head;
}

```

```

}
ptrDNODE DLIST::get_head()
{
    return head;
}
void DLIST::insert_after(ptrDNODE ptr, TInfo elem)
{
    ptrDNODE p = new DNODE(elem, ptr->next, ptr);
    if (ptr == tail)
        tail = p;
    else
        ptr->next->prev = p;
    ptr->next = p;
}
void DLIST::insert_before(ptrDNODE ptr, TInfo elem)
{
    ptrDNODE p = new DNODE(elem, ptr, ptr->prev);
    if (ptr == head)
        head = p;
    else
        ptr->prev->next = p;
    ptr->prev = p;
}
void DLIST::print()
{
    ptrDNODE p = head;
    while (p)
    {
        (p->info)->print();
        p = p->next;
    }
    std::cout << "\n";
}
DLIST::DLIST(std::ifstream & file)
{
    TInfo elem = new Book(file);
    first_node(elem);
    while (!file.eof())
    {
        elem = new Book(file);
        insert_after(tail, elem);
    }
    file.close();
}

```



```

}
DLIST::DLIST(std::ifstream & file, std::function<int(Book, Book)>compare)
{
    TInfo elem = new Book(file);
    first_node(elem);
    auto find_place = [this, compare](TInfo elem)->ptrDNODE
    {
        ptrDNODE p = head;
        while (p && compare(*(p->info), *elem))
            p = p->next;
        return p;
    };
    ptrDNODE place{};
    while (!file.eof())
    {
        elem = new Book(file);
        place = find_place(elem);
        if (place)
            insert_before(place, elem);
        else
            insert_after(tail, elem);
    }
    file.close();
}

Book::Book(std::ifstream & file)
{
    file.getline(author, m);
    file.getline(title, m);
    file >> year;
    file >> pages;
    file.ignore();
    file.getline(specialty, m);
    if (!file.eof())
    {
        char delim_file[255];
        file.getline(delim_file, 255);
    }
}

void Book::print()
{
    std::cout << author << '\n';
    std::cout << title << '\n';
    std::cout << year << '\n';

```

```

std::cout << pages << '\n';
std::cout << specialty << '\n';
std::cout << "-----\n";
}

void DLIST::remove(ptrDNODE & ptr)
{
    ptrDNODE p = ptr;
    if (ptr == head)
    {
        head = p->next;
        ptr = head;
        if (p->next)
            p->next->prev = nullptr;
    }
    else
        if (ptr == tail)
        {
            tail = p->prev;
            ptr = tail;
            if (p->prev)
                p->prev->next = nullptr;
        }
        else
        {
            ptr = ptr->next;
            p->next->prev = p->prev;
            p->prev->next = p->next;
        }
    delete p;
}

```

Заключение

Списки — это важная и полезная структура данных, обеспечивающая высокую гибкость и динамическое управление элементами. Несмотря на некоторые недостатки, такие как сложность доступа и повышенные затраты памяти, их преимущества в простоте добавления и удаления элементов делают их ценным инструментом в программировании. Примеры кода и варианты использования, представленные в этом обзоре, демонстрируют широкие возможности и применение списков в различных областях программирования.

Связанные списки удобно использовать, когда у вас есть неизвестное количество элементов для хранения. Использование структуры данных, такой как массив, потребует от вас заранее указать размер; превышение этого размера предполагает вызов алгоритма изменения размера, время работы которого линейно. Вам также следует использовать связанные списки, когда вы удаляете узлы только в начале или в конце списка, чтобы поддерживать постоянное время выполнения. Для этого необходимо поддерживать указатели на узлы в начале и в конце списка, но затраты памяти окупятся, если эту операцию вы будете выполнять много раз. Для чего связанные списки не очень хороши, так это для случайной вставки, доступа к узлам по индексу и поиска. За счет небольшого объема памяти (в большинстве случаев достаточно 4 байтов) и еще нескольких операций чтения/записи вы можете поддерживать переменную счетчика, которая отслеживает, сколько элементов содержится в списке, чтобы доступ к такому примитивному свойству был постоянным. операция — вам просто нужно обновить счетчик во время алгоритмов вставки и удаления. Односвязные списки следует использовать, когда вы выполняете только базовые вставки. В целом двусвязные списки более удобны для нетривиальных операций со связанным списком. Мы рекомендуем использовать двусвязный список, когда вам требуется обход вперед и назад. В большинстве случаев это требование присутствует. Например, рассмотрим поток токенов, который вы хотите проанализировать методом рекурсивного

спуска. Иногда вам придется вернуться назад, чтобы создать правильное дерево разбора. В этом случае лучше всего подходит двунаправленный список, поскольку его конструкция делает двунаправленный обход намного проще и быстрее, чем односвязный список.

Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. – 3-е издание. – Москва: Вильямс, 2013.
2. Double Linked lists [Электронный ресурс] – Режим доступа: <https://www.geeksforgeeks.org/introduction-and-insertion-in-a-doubly-linked-list/>
3. Роберт Седжвик. Алгоритмы на C++. – 3-е издание. – Москва: Бином, 2011.
4. Algorithms Notes for Professionals-GoalKicker.com <http://goalkicker.com/>
5. Эллис Хорват и Джон Стратус. Увлекательный C++. – 2-е издание. – Wrox Press, 2003.
6. Data Structure Flashcards [Электронный ресурс] – Режим доступа: <https://quizlet.com/315172769/data-structures-flash-cards/>
7. Operation of Doubly Linked List with Implementation [Электронный ресурс] – Режим доступа: <https://www.geeksforgeeks.org/doubly-linked-list-tutorial/>
8. Роберт Лафоре. Объектно-ориентированное программирование в C++. – 4-е издание. – Sams Publishing, 2002.
9. C++ Notes for Professionals, GoalKicker.com. Free Programming Books. <http://goalkicker.com/>
10. Programming Exercise – Doubly Linked List the purpose [Электронный ресурс] – Режим доступа: <https://www.chegg.com/homework-help/questions-and-answers/programming-exercise-doubly-linked-list-purpose-exercises-give-additional-practice-working-q116833997>
11. Дейтел П., Дейтел Х. Как программировать на C++. – 10-е издание. – Pearson, 2017.